

# Day 10 — Live coding: dai flow a un portfolio

Il Day 9 ha costruito un singolo caso d'uso dall'inizio alla fine. Oggi non costruiamo **nulla di meccanicamente nuovo** — nessun nuovo tipo di step, nessuna nuova anatomia di action. Facciamo invece crescere l'assistente da sei flow a nove e osserviamo comparire una classe di problemi diversa: flow *in competizione* tra loro. Ogni cosa difficile in questa sessione è una scelta di giudizio, non una questione di sintassi: come chiamare un flow, come descriverlo, cosa l'LLM può vedere e dove risiede lo stato. Se oggi lo YAML sembra facile, è proprio questo il punto — lo YAML è facile ormai; ciò che resta è l'architettura.

In questa lezione si lavora nella cartella `starting-point/` — il progetto esattamente com'era alla fine del Day 9. Lo stato finito è distribuito accanto ad essa in `end-result/` ( `lumera-assistant/`, il cresciuto `lumera-fastapi-server/` e il `Makefile` ); ricorri ad esso quando uno step ti dice di copiarvi dentro un file, oppure per confrontare il risultato quando hai finito.

**REQUIREMENTS** — il progetto com'era alla fine del Day 9, `RASA_LICENSE` e `OPENAI_API_KEY` esportate, `CORE_BANKING_URL=http://localhost:8000` e `CORE_BANKING_TOKEN=test_token` in qualsiasi terminale che esegua `rasa` o la mock bank (il `Makefile` imposta questi ultimi due per i processi che avvia). **Una nuova installazione:** la sezione 6 usa componenti classic-NLU, che sono un extra opzionale di Rasa Pro — il `requirements.txt` di `starting-point/` dice già `rasa-pro[nlu]>=3.17`, quindi un `make setup` da zero la copre. (Su Apple Silicon l'extra tira dentro `tensorflow-metal`, che al momento è incompatibile con il TensorFlow risolto accanto ad esso; il `Makefile` lo rimuove dopo l'installazione — non usiamo alcun componente TensorFlow, il nostro classifier è scikit-learn.)

## 1. Tre nuovi flow — il lavoro sta nelle parole

Il portfolio cresce di un **cluster di carte** — `block_card` e `replace_card` — più `list_transactions`. Osserva cosa abbiamo appena fatto: abbiamo creato due coppie di *vicini prossimi*. Blocco e sostituzione sono entrambi "cose da carta"; le transazioni e la già esistente consultazione del saldo sono entrambe "cose da conto". Il messaggio di un cliente può plausibilmente significare l'uno o l'altro membro di una coppia. Quella collisione è il tema centrale di oggi, e l'abbiamo piantata di proposito.

I meccanismi sono tutti noti. Ogni nuovo flow usa il pattern di action del Day 8 (HTTP verso la mock bank, `SlotSet` in uscita) e le decisioni di fiducia del Day 9 (confirmation gate sull'operazione distruttiva). Le tre action non contengono nulla che il Day 8 non abbia già insegnato, quindi per brevità non le mostreremo in questa guida — Usa il punto di partenza oppure, se stai proseguendo dall'ultimo live coding, copia i file `lumera-assistant/actions/block_card.py`, `lumera-assistant/actions/replace_card.py` e `lumera-assistant/actions/fetch_transactions.py` dalla `end-result/` di questa lezione nel tuo progetto. I tre mock endpoint che esse chiamano ( `POST /v1/cards/block`, `POST /v1/cards/replace`, `GET /v1/accounts/{account_type}/transactions` ) fanno già parte del server della mock bank di oggi, che esegui anziché modificare. I minuti della sessione di oggi vanno allo YAML — e per lo più alle *parole* al suo interno.

Hai portato avanti il `data/flows.yml` del Day 9 — sei flow sotto un'unica chiave `flows:` . I tre nuovi flow si **aggiungono** a quello stesso file: incolla ciascuno sotto la chiave `flows:` esistente, accanto ai flow già presenti. Ecco perché gli snippet qui sotto iniziano dal nome del flow ( `block_card:` , indentazione di due spazi) senza una loro riga `flows:` — sono frammenti da aggiungere, non file da creare. (La sezione 2 divide questo monolite in file per singolo flow; fino ad allora, tutto risiede in `data/flows.yml` .)

Ecco `block_card` , prima versione:

```
block_card:
  name: block a card
  description: Handle problems with the customer's card.
  steps:
    - call: authenticate_user
    - noop: true
    - next:
      - if: not slots.authenticated
        then:
          - action: utter_auth_failed
            next: END
        - else: ask_card
    - collect: card_type
      id: ask_card
      utter: utter_ask_card_to_block
      description: "the card to block: debit or credit"
    - collect: block_confirmed
      description: whether the customer confirms blocking the card
      ask_before_filling: true
    - next:
      - if: slots.block_confirmed
        then:
          - action: action_block_card
            next:
              - if: slots.card_block_ok
                then: card_blocked
              - else:
                - action: utter_card_block_failed
                  next: END
        - else:
          - action: utter_card_block_declined
            next: END
    - action: utter_card_blocked
      id: card_blocked
```

Tre decisioni deliberate qui dentro:

- **call: authenticate\_user è il primo step.** Bloccare una carta è distruttivo, quindi riceve lo stesso guardiano che il trasferimento ha ricevuto nel Day 9 — scritto una volta, riutilizzato. Questo è il dividendo del subflow callable-only: la seconda operazione sensibile costa una riga.
- **Il confirmation gate** (`ask_before_filling: true` su `block_confirmed`) — il blocco è irreversibile dalla chat, quindi il consenso viene raccolto ex novo, ogni volta, esattamente come

il recap del trasferimento.

- **utter: utter\_ask\_card\_to\_block** — uno step **collect** normalmente chiede tramite **utter\_ask\_<slot\_name>**; la proprietà **utter:** ci permette di riutilizzare lo stesso slot **card\_type** che **card\_fees\_info** possiede già, ponendo però una domanda adatta a questo flow ("Quale carta hai bisogno di bloccare?"). Uno slot, formulazione per singolo flow.

E la description? "*Handle problems with the customer's card.*" — ce ne pentiremo nella sezione 3, di proposito. (Come sempre, la cartella **end-result/** contiene lo *stato finale* — le description corrette e il layout della sezione 2. Per vivere il percorso, digita le versioni mostrate qui e correggile quando lo faremo noi.)

Ora **replace\_card**, aggiunto subito dopo, sotto la stessa chiave **flows:** :

```
replace_card:
  name: replace a card
  description: Help the customer with a card issue.
  steps:
    - collect: card_type
      utter: utter_ask_card_to_replace
      description: "the card to replace: debit or credit"
    - collect: replace_confirmed
      description: whether the customer confirms ordering the replacement card
      ask_before_filling: true
      next:
        - if: slots.replace_confirmed
          then:
            - action: action_replace_card
              next:
                - if: slots.card_replace_ok
                  then: card_replaced
                - else:
                  - action: utter_card_replace_failed
                    next: END
          - else:
            - action: utter_card_replace_declined
              next: END
        - action: utter_card_replacement_ordered
      id: card_replaced
```

Stesso scheletro — con **nessun call: authenticate\_user**. Questa è una decisione, non un'omissione: una carta sostitutiva viene spedita all'indirizzo registrato, quindi il danno che un estraneo può fare ordinandone una è modesto, e abbiamo scelto di non gravare il cliente con un codice per essa. Il blocco, al contrario, uccide istantaneamente una carta funzionante. Le *decisioni di fiducia per singolo flow* sono progettazione di portfolio: i due flow sembrano gemelli e differiscono deliberatamente in esattamente un modo strutturale. (Il team di sicurezza della tua banca potrebbe decidere diversamente — il punto è che si tratta di una decisione per singolo flow, presa consapevolmente.)

**list\_transactions** è la forma di **check\_balance** con una diversa lettura del backend — stesso slot **account\_type**, action di fetch, branch sullo status-flag e una risposta che stampa le righe che

l'action ha formattato in uno slot `transactions_list`. La sua prima description: *"Show the customer what is in their account."* Anch'essa deprecabile, anch'essa di proposito — e, come le altre due, aggiunta sotto la chiave `flows:` esistente:

```
list_transactions:
  name: list recent transactions
  description: Show the customer what is in their account.
  steps:
    - collect: account_type
      description: "the account whose transactions to list: checking or savings"
    - action: action_fetch_transactions
      next:
        - if: slots.transactions_fetch_ok
          then: show_transactions
        - else:
            - action: utter_transactions_service_down
              next: END
    - action: utter_recent_transactions
  id: show_transactions
```

I file del domain contengono i nuovi slot — i due bool di conferma, i flag di status `controlled`, `card_replacement_eta`, `transactions_list` — più le ask con button e le risposte di esito. Tutto materiale del Day 7–8, tre nuovi file. `domain/block_card.yml`:

```

version: "3.1"

slots:
  block_confirmed:
    type: bool
    mappings:
      - type: from_llm
  card_block_ok:
    type: bool
    mappings:
      - type: controlled

responses:
  utter_ask_card_to_block:
    - text: "Which card do you need to block?"
      buttons:
        - title: "Debit card"
          payload: "/SetSlots(card_type=debit)"
        - title: "Credit card"
          payload: "/SetSlots(card_type=credit)"

  utter_ask_block_confirmed:
    - text: "I'm about to block your {card_type} card immediately – it can no longer be
      used after this. Shall I proceed?"
      buttons:
        - title: "Yes, block it"
          payload: "/SetSlots(block_confirmed=true)"
        - title: "No, don't"
          payload: "/SetSlots(block_confirmed=false)"

  utter_card_blocked:
    - text: "Done – your {card_type} card is now blocked. If you also need a
      replacement, just ask."

  utter_card_block_failed:
    - text: "I couldn't reach the card system, so the card has NOT been blocked. Please
      call the emergency number on our website if this is urgent."

  utter_card_block_declined:
    - text: "Understood – your {card_type} card stays active."

actions:
  - action_block_card

```

domain/replace\_card.yml:

```

version: "3.1"

slots:
  replace_confirmed:
    type: bool
    mappings:
      - type: from_llm
  card_replace_ok:
    type: bool
    mappings:
      - type: controlled
  card_replacement_eta:
    type: text
    mappings:
      - type: controlled

responses:
  utter_ask_card_to_replace:
    - text: "Which card should I order a replacement for?"
      buttons:
        - title: "Debit card"
          payload: "/SetSlots(card_type=debit)"
        - title: "Credit card"
          payload: "/SetSlots(card_type=credit)"

  utter_ask_replace_confirmed:
    - text: "I'll order a replacement {card_type} card, delivered to your registered address. Shall I proceed?"
      buttons:
        - title: "Yes, order it"
          payload: "/SetSlots(replace_confirmed=true)"
        - title: "No, don't"
          payload: "/SetSlots(replace_confirmed=false)"

  utter_card_replacement_ordered:
    - text: "Done – your replacement {card_type} card is on its way and should arrive within {card_replacement_eta}."

  utter_card_replace_failed:
    - text: "I couldn't reach the card system, so no replacement has been ordered. Please try again later."

  utter_card_replace_declined:
    - text: "Understood – I won't order a replacement."

actions:
  - action_replace_card

```

domain/list\_transactions.yml:

```

version: "3.1"

slots:
  transactions_list:
    type: text
    mappings:
      - type: controlled
  transactions_fetch_ok:
    type: bool
    mappings:
      - type: controlled

responses:
  utter_recent_transactions:
    - text: "Here are the latest movements on your {account_type}
account:\n{transactions_list}"

  utter_transactions_service_down:
    - text: "We're sorry, the transactions service is unavailable right now. Please try
again later."

actions:
  - action_fetch_transactions

```

Nota cosa *non* è dichiarato qui: `card_type`. Entrambi i card flow lo raccolgono, ma esso appartiene già al `card_fees_info.yml` del Day 7 — gli slot sono a livello di intera conversazione, e la formulazione `utter:` per singolo flow è ciò che fa sentire lo slot condiviso come nativo di ciascun flow.

Riavvia la mock bank — esegui `make run-fastapi-server` (il server di `starting-point/` porta già i tre nuovi endpoint di oggi) — poi allena e prova uno:

```

user: show me my recent transactions bot: Which account would you like to check? user:
checking bot: Here are the latest movements on your checking account: 2026-06-27 Salary —
Rossi & Partners +€2450.00 2026-06-28 Supermarket Esse-Piu -€74.30 2026-06-30 Card
payment — Trattoria da Peppe -€38.50 2026-07-01 Utility bill — electricity -€61.20 2026-07-03
ATM withdrawal -€100.00

```

(Una raffinatezza nel mock: i trasferimenti eseguiti vengono aggiunti alla lista del conto checking — invia denaro e comparirà qui, proprio come la detrazione dal saldo del Day 9.)

## 2. Nove flow vogliono un filesystem — un flow per file

`data/flows.yml` è ora lungo nove flow, e il quarantesimo flow è in arrivo. Prima che lo faccia, imponiamo il layout su scala: **un flow per file, raggruppati per topic** — e mandiamo in pensione `flows.yml`:

```
data/flows/
  accounts/      check_balance.yml, list_transactions.yml, download_statement.yml
  cards/         block_card.yml, replace_card.yml, card_fees_info.yml
  transfers/     transfer_money.yml
  appointments/  book_appointment.yml
  shared/        authenticate_user.yml
```

Onestà prima di tutto: il *domain* è suddiviso per topic fin dal Day 7 — oggi completiamo semplicemente la stessa disciplina sul lato dei flow. Rasa legge tutto ciò che si trova sotto `data/` ricorsivamente, quindi lo spostamento è pura chirurgia sui file: taglia ogni voce `flows:` nel proprio file (ciascun file inizia con la propria chiave `flows:` ), cancella il monolite e dimostra che è stato gratuito:

```
rasa data validate
rasa train
```

Stesso assistente, zero cambiamenti di comportamento. Ciò che abbiamo comprato è la convenzione che scala: **nomi che si greppano**. Il flow `block_card` risiede in `block_card.yml`, chiede tramite risposte `utter_ask_...` nominate secondo i suoi slot, è implementato da `action_block_card` — così `grep -r block_card` risponde a "cosa succede quando un cliente blocca una carta?" attraverso YAML, domain e Python in un colpo solo. Quando il portfolio raggiunge quaranta flow, quella convenzione è la documentazione.

### 3. Steering: flow retrieval attivo, e le description diventano contrasti

Apri `config.yml`. Dal Day 7 porta:

```
flow_retrieval:
  active: false
```

Abbiamo disattivato il retrieval quando avevamo tre flow e ogni token di prompt era facile da ragionare. **Cancella quelle due righe** — a nove flow e in crescita, la riga si guadagna il suo posto. Cosa fa: al momento del training, la description di ogni flow viene embeddata in un vettore; a runtime anche il messaggio dell'utente viene embeddato, e solo i flow *più simili* vengono messi nel prompt dell'LLM. I costi, onestamente: il training ora effettua chiamate di embedding (la tua `OPENAI_API_KEY` viene usata al momento del training), e ogni turno embedda il messaggio in ingresso. Ciò che compra: il prompt non cresce più linearmente con il portfolio — con centinaia di flow, l'LLM legge comunque solo una shortlist.

Alla nostra dimensione la shortlist di default (top 20) includerebbe *tutto*, quindi per **vedere** davvero il retrieval al lavoro, rendilo temporaneamente aggressivo:

```
flow_retrieval:
  num_flows: 4
```



Riallana, ed esegui il server con il prompt del command generator reso visibile:

```
LOG_LEVEL_LLM_COMMAND_GENERATOR=INFO rasa inspect
```

Digita un messaggio ambiguo — "something is wrong with my card" — e leggi il terminale. Il prompt renderizzato contiene una lista JSON di flow candidati, ed ha esattamente quattro voci:

```
{"flows": [
  {"name": "card_fees_info", "description": "Explain the yearly fees of Banca Lumera payment cards.", "..."},
  {"name": "replace_card", "description": "Help the customer with a card issue.", "..."},
  {"name": "block_card", "description": "Handle problems with the customer's card.", "..."},
  {"name": "check_balance", "description": "Look up the current balance of one of the customer's Banca Lumera accounts.", "..."}
]}
```

Due cose da leggere in questo, e la prima risponde a una domanda che questa classe si è già posta in passato:

- **Il retrieval non può scegliere un flow.** Non c'è alcuna soglia di confidenza da configurare, e nessun percorso da "vettore più vicino" ad "avvia il flow". Il retrieval decide soltanto *chi è nella stanza*; il command generator — l'LLM — prende ancora la decisione, e puoi osservarlo farlo nella riga di log immediatamente successiva. Se speravi di instradare sulla sola similarità vettoriale e saltare l'LLM: questo componente non lo farà mai. (La sezione 6 mostra il meccanismo che *invece* salta l'LLM.)
- **authenticate\_user non è nella lista — e non lo sarà mai.** Il suo guard `if: False` del Day 9 lo esclude interamente dal prompt, a qualsiasi `num_flows`. Un flow che il modello non può vedere è un flow che il modello non può avviare e che nessuna prompt injection può raggiungere: i guard sono un confine di *sicurezza*, non solo una comodità di routing. (Non farti tentare, però, dall'appiccicare `if: slots.authenticated` su `block_card` stesso — un flow guardato è invisibile anche agli utenti *freschi*, quindi nessuno potrebbe mai chiedere di bloccare una carta. La forma del Day 9 è quella giusta: il flow resta avviabile, e il `call` protegge la parte sensibile *all'interno*.)

Ora la collisione che abbiamo piantato. Lo stesso log mostra cosa l'LLM ha fatto con i nostri quattro candidati:

```
action_list: "disambiguate flows replace_card block_card block_card"
```

E nella chat:

**user:** something is wrong with my card **bot:** I can help, but I need more information. Which of these would you like to do: block a card, block a card or replace a card?

Il motore ha fatto la cosa giusta — di fronte a un messaggio che genuinamente si adatta a diversi flow, si è rifiutato di indovinare e ha chiesto. Questo è il **clarification pattern** integrato che fa il suo lavoro, e *non* lo personalizzeremo (i pattern hanno una loro giornata dedicata). Ma guarda più da vicino la qualità della domanda: "block a card" è offerto **due volte**. Le nostre due description — "Handle problems with the customer's card" / "Help the customer with a card issue" — non hanno dato al modello nulla con cui separare i flow, e il pasticcio si palesa nella clarification stessa.

La correzione è a monte, nelle parole. **Scrivi le description come contrasti** — dì per cosa il flow *non* è, ogni volta che un vicino potrebbe confondere:

```
block_card:
  description: >
    Immediately block one of the customer's Banca Lumera cards that is lost,
    stolen or compromised, so it can no longer be used. Does not order a
    replacement card.

replace_card:
  description: >
    Order a replacement for one of the customer's Banca Lumera cards that is
    damaged, expired or already blocked. Does not block an active card.
```

E la coppia dei conti:

```
check_balance:
  description: >
    Look up the current balance of one of the customer's Banca Lumera
    accounts. Does not list past transactions.

list_transactions:
  description: >
    List the most recent transactions on one of the customer's Banca Lumera
    accounts. Does not show the current account balance.
```

Le frasi "Does not..." funzionano due volte: allontanano i vicinati di embedding delle due description (retrieval), e consegnano all'LLM il confine esatto (selezione). Riallena e ripeti:

**user:** something is wrong with my card **bot:** I can help, but I need more information. Which of these would you like to do: block a card, card fees information or replace a card?

Ancora una clarification — *correttamente*: quel messaggio non porta alcun segnale su quale operazione sulla carta si intenda, e chiedere è la risposta onesta. Ma la domanda è ora ben formata: tre opzioni reali, distinte, senza duplicati. E i messaggi che *invece* portano segnale ora vengono instradati senza una domanda:

**user:** I lost my card, someone might use it **bot:** For your security, please enter your one-time code. (→ *block\_card*, dritto attraverso il suo auth gate)

**user:** my card is damaged, I need a new one **bot:** Which card should I order a replacement for?  
(→ `replace_card` — e nota: nessun codice richiesto. L'asimmetria della sezione 1, resa visibile.)

Una nota di onestà: un LLM potente spesso instrada correttamente i messaggi ovvi anche con description sciatte — non lasciare che ciò ti culli. Scrivi contrasti per i casi di confine, per la qualità delle clarification e per il retrieval a una dimensione di portfolio in cui il modello non vede più ogni flow. Le description costano poco; revisionale **come insieme**, dando la caccia alle sovrapposizioni nello stesso modo in cui revisioni le route di un'API alla ricerca di collisioni.

Prima di proseguire, rimuovi la riga `num_flows: 4` — era un espediente dimostrativo, non architettura. Il retrieval resta **attivo** con i suoi default d'ora in avanti.

## 4. Una digressione, gestita da macchinari che già possiedi

I cambi di argomento a metà compito sono il modo in cui gli esseri umani parlano. Avvia un trasferimento e, a metà, chiedi qualcos'altro:

**user:** send 50 euros to Marco Bassi **bot:** For your security, please enter your one-time code.  
**user:** 123456 **bot:** What is the recipient's IBAN? **user:** IT12A0300203280123456789012 **bot:**  
Sorry, it seems the value you provided `50 euros` is not a valid number. Please provide a valid number in your response. **bot:** How much would you like to transfer, in euros? **user:** wait, what is my balance? **bot:** Which account would you like to check? **user:** the checking one **bot:** Your checking account balance is €1250.0. **bot:** Would you like to continue with transfer money? **user:** yes **bot:** How much would you like to transfer, in euros? **user:** 50 **bot:** Please confirm: transfer €50.0 to Marco Bassi, IBAN IT12A0300203280123456789012. Shall I proceed?

(Lo scambio "not a valid number" nel mezzo è lo slot di tipo float che rifiuta per tipo il testo letterale "50 euros", esattamente come nel Day 9 — la riga di validazione gratuita che fa il suo lavoro ancora prima che la digressione inizi.)

Leggi cosa è successo a "wait, what is my balance?": il command generator ha avviato `check_balance`, che è stato **spinto sopra** al `transfer_money` in pausa; è arrivato alla sua fine ed è stato estratto (pop); poi un pattern integrato (`pattern_continue_interrupted`) ha riorferto il trasferimento — con il destinatario, l'IBAN e tutto il resto ancora compilati. Osservalo nel pannello dello stack dell'Inspector: due flow impilati durante la deviazione, esattamente come il `call` del Day 9 — solo che **nessuno ha scritto un `call`**. Lo stack è del motore; le digressioni arrivano gratis.

Ciò che *non* è gratis è renderle piacevoli, e questa è di nuovo progettazione di portfolio: flow piccoli e monomansione e description oneste fanno costare la deviazione al cliente una sola domanda, non un reinserimento di tutto. Una consultazione del saldo che interrompe un trasferimento è utile (potrebbero star verificando di poterselo permettere); progetta i tuoi flow in modo che gli interruttori a cui le persone effettivamente ricorrono siano economici. I repair pattern che sono scattati oggi — clarification nella sezione 3, continue-interrupted qui — sono integrati, nominati e personalizzabili; *quando* personalizzarli è la domanda del Day 11, non di oggi.

## 5. Scoping dello stato — una sfida per sessione

Il Day 9 si è chiuso su un anticipo: chiedi un secondo trasferimento e il codice viene richiesto di nuovo. Ora morde davvero. Completa un trasferimento (codice richiesto, va bene), poi:

**user:** now block my debit card please **bot:** For your security, please enter your one-time code.

Due sfide in una sola sessione. Per l'app autenticata di una banca questa è cattiva UX — il cliente ha dimostrato chi è novanta secondi fa.

Per correggerlo come si deve, impara le **regole di scoping** — dove i valori degli slot vivono e muoiono:

1. **Locale al flow (il default):** gli slot compilati da step `collect` o `set_slots` vengono **resettati quando il loro flow termina**. L'importo di un trasferimento non dovrebbe trapelare nel topic successivo; il default è quello giusto.
2. **Persistito oltre il flow:** un flow può esentare specifici slot compilati da `collect/set` dal reset elencandoli nella sua proprietà `persisted_slots`. (Solo gli slot compilati da `collect/set_slots` vi appartengono — elencare uno slot compilato da action è un errore in fase di training.)
3. **Gli slot impostati da custom action persistono da soli.** Nessun reset si applica ad essi al termine del flow.
4. **Attraverso le sessioni:** `session_config.carry_over_slots_to_new_session` nel domain decide cosa sopravvive a un confine di sessione — un asse separato che già portiamo in `domain/shared.yml`.

La regola 3 è la sorpresa, e spiega con precisione il nostro bug. `authenticated` è impostato da `action_verify_customer` — uno slot impostato da action — quindi **sopravvive già** al termine di `authenticate_user` e del trasferimento. Lo stato è lì; guarda il pannello memory dell'Inspector dopo il trasferimento e vedrai `authenticated: true` seduto al suo interno. La seconda sfida accade perché il `call` riesegue il child flow, e il child *chiede incondizionatamente il codice* (il suo `collect` è `ask-always`, per design stesso del Day 9). La correzione non è quindi la persistenza — la correzione è insegnare al guardiano a controllare prima di lanciare la sfida. Un branch in cima a `data/flows/shared/authenticate_user.yml`:

```
flows:
  authenticate_user:
    if: False
    name: verify customer identity
    description: Verify the customer's identity with a one-time security code before a
sensitive operation.
    steps:
      - noop: true
        next:
          - if: slots.authenticated
            then: END
          - else: ask_code
      - collect: security_code
        id: ask_code
        description: the one-time security code the customer provides
        ask_before_filling: true
      - action: action_verify_customer
```

Già verificato → il child termina immediatamente → il `call` del parent ritorna nello stesso turno.  
Riallena e ripeti la coppia:

**user:** *(transfer completed, code asked once)* now block my debit card please **bot:** I'm about to block your debit card immediately — it can no longer be used after this. Shall I proceed?

Dritto al consent gate. Una sfida per sessione — e nota che *ogni* flow sensibile presente e futuro eredita la correzione, perché tutti `call` ano lo stesso guardiano.

Di il contrappeso di sicurezza ad alta voce prima che qualcuno lo chieda: l'autenticazione a livello di sessione è una **scelta di policy**, non un'inevitabilità tecnica. La ri-autenticazione per operazione è una policy difendibile per i pagamenti; se il tuo team di sicurezza la vuole, il comportamento del Day 9 era già corretto. Se adotti l'autenticazione a livello di sessione, decidi come *scade*: un deliberato `set_slots: [authenticated: null]` ovunque la policy lo dica (un'action di timeout, un flow di logout). E controlla l'asse cross-session: `carry_over_slots_to_new_session: true` è in `shared.yml` dal Day 7 — per una banca reale valuteresti se `authenticated` possa sopravvivere alla sessione del tutto. Lo stato che sopravvive alla propria giustificazione è una passività, non una comodità.

Dove `persisted_slots` è lo strumento giusto: uno slot compilato da collect che un flow fratello può onestamente riutilizzare. La nostra coppia: consulti il saldo del tuo conto savings, poi chiedi i movimenti — l'assistente dovrebbe richiedere "quale conto?" Lo sa già. In `check_balance.yml`:

```
check_balance:
  persisted_slots:
    - account_type
```

Riallena:

**user:** check my savings balance **bot:** Your savings account balance is €2087.5. **user:** and show me the transactions on it **bot:** Here are the latest movements on your savings account: 2026-06-15 Transfer from checking +€200.00 2026-06-30 Interest credit +€3.40

Nessuna riproposizione della domanda — ma sii preciso sul *perché*, perché abbiamo testato entrambe le versioni e il transcript da solo non te lo dice. Osserva il pannello memory dell'Inspector nel momento in cui il flow del saldo termina: **con** `persisted_slots`, `account_type: savings` è ancora lì seduto; **senza**, il valore viene cancellato in quell'istante (regola 1). In questa particolare conversazione la versione non persistita *ugualmente* si è trovata a non richiedere la domanda — l'LLM ha ri-derivato "savings" dal messaggio precedente da solo. Questa è esattamente la differenza che conta: senza persistenza ti affidi alla buona volontà contestuale del modello, che varia con la formulazione, la distanza e la versione del modello; con `persisted_slots` il valore è *strutturalmente* in memoria, deterministicamente, per ogni flow che viene dopo. L'intero tema di oggi in un solo slot: lo stato a cui tieni riceve uno scope dichiarato — non è lasciato all'inferenza. Ed è questo il giudizio da esprimere flow per flow: *questo slot è un valore di lavoro (lascialo morire) o è stato di sessione (persistilo, e scrivi il perché)?*

## 6. Extra — il sidecar NLU: avviare flow senza l'LLM

---

Come esercizio extra, vediamo come possiamo usare un classifier NLU classico per avviare flow senza invocare affatto l'LLM.

Tre pezzi. Primo, dati di training NLU classici — un nuovo `data/nlu.yml` :

```

nlu:
- intent: check_balance
  examples: |
    - what's my balance
    - show me my balance
    - how much money do I have
    - check my account balance
    - what is my current balance
    - how much is in my checking account
    - how much is in my savings account
    - balance please
    - can you check my balance
    - what's the balance on my account

- intent: list_transactions
  examples: |
    - show me my recent transactions
    - what are my latest transactions
    - list my account movements
    - what did I spend recently
    - show the last movements on my account
    - recent activity on my checking account
    - what transactions went through my savings account
    - show my transaction history
    - list my recent payments
    - what happened on my account lately

```

Perché *due* intent quando ne cabliamo uno solo? Un classifier si rifiuta di allenarsi su una singola classe — `rasa train` fallisce al nodo di training del classifier con un solo intent definito. Due è il minimo; `list_transactions` fornisce inoltre al classifier un contrasto realistico per i messaggi di tipo account. Dichiarali entrambi nel domain — un nuovo `domain/nlu.yml`:

```

version: "3.1"

intents:
- check_balance
- list_transactions

```

Secondo, la pipeline in `config.yml` cresce un front end classico:

```

pipeline:
- name: WhitespaceTokenizer
- name: CountVectorsFeaturizer
- name: LogisticRegressionClassifier
- name: NLUCommandAdapter
- name: CompactLLMCommandGenerator
llm:
  model_group: openai_llm

```

Tokenizer → featurizer → intent classifier è lo stack classic-NLU più piccolo (regressione logistica scikit-learn — veloce, nessun deep learning); l' `NLUCommandAdapter` è il ponte: legge l'intent predetto e, se qualche flow dichiara un trigger corrispondente, emette esso stesso il comando di avvio di quel flow. (Questi componenti sono il motivo per cui `requirements.txt` ha guadagnato l'extra `[nlu]` .)

Terzo, il trigger, sul flow — in `check_balance.yml` :

```
check_balance:
  nlu_trigger:
    - intent:
        name: check_balance
        confidence_threshold: 0.75
```

**La soglia è misurata, non indovinata.** Esegui `rasa run --enable-api` e fai POST di qualche messaggio a `/model/parse` per vedere cosa dice effettivamente il classifier:

```
curl -X POST http://localhost:5005/model/parse -H "Content-Type: application/json" -d '{"text": "what is my balance"}'
```

| message                          | predicted intent  | confidence |
|----------------------------------|-------------------|------------|
| "what is my balance"             | check_balance     | 0.81       |
| "show me my recent transactions" | list_transactions | 0.83       |
| "block my debit card"            | list_transactions | 0.58       |
| "123456"                         | list_transactions | 0.55       |

Le formulazioni reali di saldo segnano ~0.80; tutto il resto che questo minuscolo modello a due classi deve comunque etichettare atterra vicino a 0.55. Quindi 0.75 separa pulitamente — mentre un "rassicurante" 0.9 semplicemente non scatterebbe mai, e il sidecar sarebbe decorazione. (Nota che il classifier etichetta allegramente "block my debit card" come `list_transactions` — un modello classic-NLU risponde *sempre* con qualcosa dalla sua lista di intent; la soglia è ciò che impedisce a quella sciocchezza di agire.)

Riallena e osserva i log ( `LOG_LEVEL_NLU_COMMAND_ADAPTER=INFO`  
`LOG_LEVEL_LLM_COMMAND_GENERATOR=INFO` ):

**user:** what's my balance **bot:** Which account would you like to check?

```
nlu_command_adapter.predict_commands {"commands":
  ["StartFlowCommand(flow='check_balance')"]}
```

...e **nessuna riga di prompt segue** — il flow è partito con zero chiamate LLM in quel turno. Questo è il parametro `minimize_num_calls` del command generator (default `true`): quando l'adapter ha già risolto il turno, l'invocazione dell'LLM viene saltata interamente. Ora il turno successivo:



**user:** checking **bot:** Your checking account balance is €1250.0.

```
nlu_command_adapter.predict_commands {"commands": []}  
llm_command_generator ... "action_list": "set slot account_type checking"
```

L'adapter può avviare flow tramite intent e riempire slot solo da segnali NLU classici (entity/intent) — non può riempire il nostro slot `from_llm`, quindi in questo turno l'LLM rientra in gioco come fallback. I due sistemi di comprensione si alternano turno per turno, prima il più economico. Ogni altro messaggio nell'assistente — carte, trasferimenti, appuntamenti — prende ancora il percorso LLM, intatto.

Un costo onesto: prova *"check my savings balance"*. L'intent scatta, l'LLM viene saltato — e la parola **"savings" viene persa**, perché il componente che l'avrebbe estratta non è mai stato eseguito; il flow richiede di nuovo quale conto. Sul percorso puro LLM quel messaggio riempiva lo slot nello stesso turno. Questo è il compromesso che `minimize_num_calls` fa: sui turni di trigger paghi zero token LLM e perdi l'estrazione di slot dell'LLM. Se i messaggi che colpiscono i tuoi trigger portano abitualmente valori di slot, imposta `minimize_num_calls: false` sul command generator — entrambi i componenti allora girano ad ogni turno (i comandi dell'adapter vincono qualsiasi conflitto) e ricompri l'estrazione al prezzo della chiamata che stavi risparmiando.

**Un caveat di sicurezza, non negoziabile:** i flow guard **non** proteggono gli avvii NLU-triggered. I guard trattengono il command generator; `call`, `link`, e `nlu_trigger` li scavalcano tutti. Un flow che è guard-gated per sicurezza e porta anche un `nlu_trigger` è un authentication bypass in attesa di essere trovato. La regola: non mettere mai un `nlu_trigger` su un flow del cui guard fai affidamento. `check_balance` è sicuro su entrambi i fronti — non ha guard, e non espone nulla di distruttivo. Se mai avessi bisogno che un flow triggered sia sicuro sotto *ogni* percorso di attivazione, esegui il controllo *all'interno* del flow, nel modo in cui i nostri flow sensibili già `call` ano il guardiano.

Due note conclusive sul quadro più ampio della coesistenza, raccontate anziché digitate. Se hai bisogno di eseguire un intero *assistente NLU legacy* — intent, rule, story, anni di tuning — accanto a CALM in un unico progetto, il setup di coesistenza di Rasa lo fa con un **router** davanti (un `IntentBasedRouter` che instrada sull'intent del classifier a costo zero aggiuntivo, o un `LLMBasedRouter` che spende una chiamata per instradare formulazioni libere); la decisione viene registrata per conversazione in uno slot e vi rimane, così un processo termina sul lato che l'ha iniziato. Questa è un'architettura di migrazione, e resta fuori dal nostro snapshot. E oltre la scala del singolo assistente, la decomposizione ha un altro asse ancora — i flow possono delegare a sub agent, e gli assistenti possono parlare con altri assistenti — che questo corso tratta più avanti; oggi, un solo assistente possiede l'intero portfolio.

## 7. Dove siamo arrivati

```
lumera-assistant/
├─ actions/
│   └─ ... (Day 8–9 actions, unchanged)
│       └─ block_card.py, replace_card.py, fetch_transactions.py    # NEW – Day 8 pattern
├─ config.yml                # retrieval ON, classic-NLU sidecar in the pipeline
├─ data/
│   └─ nlu.yml                # NEW – two intents for the sidecar
│       └─ flows/             # NEW layout – one flow per file, per topic
│           └─ accounts/ cards/ transfers/ appointments/ shared/
└─ domain/                   # + block_card, replace_card, list_transactions, nlu
lumera-fastapi-server/       # + /v1/cards/block, /v1/cards/replace,
                             #   /v1/accounts/{type}/transactions
```

Nove flow, e i veri deliverable della giornata sono invisibili in un albero dei file: description scritte come **contrasti**, un layout i cui **nomi si greppano**, guard letti come un **confine di sicurezza**, stato a cui è dato uno **scope di proposito**, e un LLM che non è più l'unico — né persino il primo — modo in cui un flow si avvia. Nulla oggi è stato un nuovo meccanismo; tutto è stato architettura.

Una cosa che *non* abbiamo fatto: quando la clarification è scattata, quando il pattern di digressione ha riorferto il trasferimento — abbiamo osservato il comportamento di repair integrato salvarci, e l'abbiamo lasciato di serie. Nella prossima sessione smettiamo di essere gentili con l'assistente: rompiamo il trasferimento in sei modi di proposito, nominiamo il pattern che raccoglie ogni caduta, e poi congeliamo l'intero percorso a ostacoli in test.